

LAPORAN PRAKTIKUM

STRUKTUR DATA

MODUL 02

PENGENALAN BAHASA C++ (BAGIAN 2)



**Universitas
Telkom**

Disusun oleh:

ERIDAYALMA ZAHRA YOHAR

109082500221

S1 IF13-03

Dosen Pengampu:

Rakhmad Maulidi, S.Kom., M.Kom

**PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS DIREKTORAT KAMPUS PURWOKERTO
UNIVERSITAS TELKOM**

2026

MODUL 2

PENGENALAN BAHASA C++ (BAGIAN 2)

1.

a. Code

```
#include <iostream>
using namespace std;

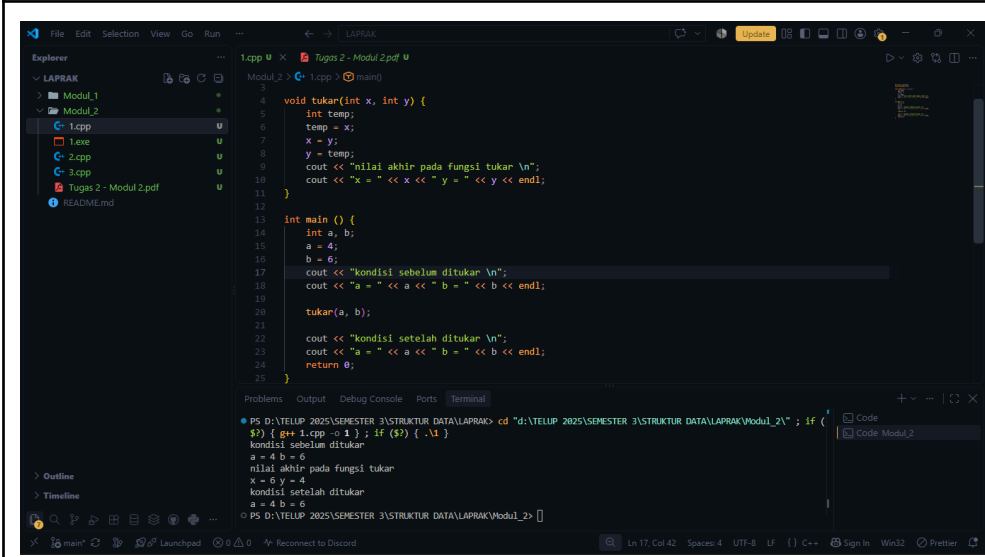
void tukar(int x, int y) {
    int temp;
    temp = x;
    x = y;
    y = temp;
    cout << "nilai akhir pada fungsi tukar \n";
    cout << "x = " << x << " y = " << y << endl;
}

int main () {
    int a, b;
    a = 4;
    b = 6;
    cout << "kondisi sebelum ditukar \n";
    cout << "a = " << a << " b = " << b << endl;

    tukar(a, b);

    cout << "kondisi setelah ditukar \n";
    cout << "a = " << a << " b = " << b << endl;
    return 0;
}
```

b. Screenshot output



The screenshot shows the Visual Studio Code editor with a C++ program. The Explorer panel on the left shows the project structure with files like 1.cpp, 1.exe, 2.cpp, 3.cpp, and Tugas 2 - Modul 2.pdf. The main editor displays the source code, which is identical to the code provided in the previous block. The Output panel at the bottom shows the execution results: "kondisi sebelum ditukar", "a = 4 b = 6", "nilai akhir pada fungsi tukar", "x = 6 y = 4", "kondisi setelah ditukar", and "a = 4 b = 6". The status bar at the bottom indicates the file is at line 17, column 42, using UTF-8 encoding and C++ language.

c. Penjelasan dari code dan output (korelasi)

Program ini *Pass by Value*, pakai fungsi tukar() yang dimana parameternya cuma int x dan int y biasa (bukan pointer atau reference). Nah, di dalam C++ itu, kalau kita kirim parameter dengan cara biasa kayak gini namanya pass by value, artinya yang dikirim ke fungsi cuma salinan dari nilai a dan b, bukan variabel aslinya.

Makanya pas di dalam fungsi tukar() nilai x dan y berhasil ditukar (kelihatan dari output "nilai akhir pada fungsi tukar" yang nunjukin x = 6, y = 4), tapi begitu balik lagi ke main(), nilai a dan b tetap sama kayak awal (a = 4, b = 6). Ini karena perubahan yang terjadi di x dan y cuma berlaku di "dunia"-nya fungsi tukar() doang, ngga nyambung sama sekali ke variabel a dan b yang ada di main(). Jadi bisa disimpulkan, pass by value itu aman buat manipulasi data sementara di dalam fungsi, tapi ngga bisa dipakai kalau tujuannya emang mau ngubah nilai variabel aslinya.

2.

a. Code

```
#include <iostream>

using namespace std;

void tukar (int *x, int *y) {
    int temp;
    temp = *x;
    *x = *y;
    *y = temp;
    cout << "nilai akhir pada fungsi tukar \n";
    cout << "x = " << *x << " y = " << *y << endl;
}

int main () {
    int a, b;
    a = 4;
    b = 6;
    cout << "kondisi sebelum ditukar \n";
    cout << "a = " << a << " b = " << b << endl;

    tukar(&a, &b);

    cout << "kondisi setelah ditukar \n";
```

```

    cout << "a = " << a << " b = " << b << endl;
    return 0;
}

```

b. *Screenshot output*

```

// 2.cpp
void tukar (int *x, int *y) {
    int temp;
    temp = *x;
    *x = *y;
    *y = temp;
    cout << "nilai akhir pada fungsi tukar \n";
    cout << "x = " << *x << " y = " << *y << endl;
}

int main () {
    int a, b;
    a = 4;
    b = 6;
    cout << "kondisi sebelum ditukar \n";
    cout << "a = " << a << " b = " << b << endl;

    tukar(&a, &b);

    cout << "kondisi setelah ditukar \n";
    cout << "a = " << a << " b = " << b << endl;
    return 0;
}

PS D:\TELUUP 2025\SEMESTER 3\STRUKTUR DATA\LAPRAK\Modul_2> cd "d:\TELUUP 2025\SEMESTER 3\STRUKTUR DATA\LAPRAK\Modul_2"
g++ 2.cpp -o 2.exe
a = 4 b = 6
nilai akhir pada fungsi tukar
x = 6 y = 4
kondisi setelah ditukar
a = 6 b = 4

```

c. Penjelasan dari code dan output (korelasi)

Program ini Pass by Pointer, yang pakai fungsi tukar() dengan parameter berupa pointer, yaitu `int *x` dan `int *y`. Beda dengan pass by value, yang dikirim ke fungsi bukan nilai `a` dan `b` secara langsung, tapi alamat memori dari variabel tersebut menggunakan `&a` dan `&b`.

Nah, karena yang dikirim adalah alamat memori, di dalam fungsi tukar() kita bisa mennggases nilai dari variabel asli menggunakan proses dereference, yaitu `*x` dan `*y`. Jadi, ketika nilai `*x` dan `*y` ditukar, yang berubah bukan cuma salinan nilai di dalam fungsi, tetapi nilai dari variabel `a` dan `b` yang ada di `main()` juga ikut berubah.

Makanya setelah fungsi tukar(`&a`, `&b`) dijalankan, nilai `a` dan `b` di dalam `main()` menjadi tertukar, yaitu `a = 6` dan `b = 4`. Ini yang membedakan pass by pointer dengan pass by value. Kalau pass by value hanya mengubah salinan nilai di dalam fungsi, pass by pointer bisa mengubah nilai variabel aslinya karena fungsi mendapatkan alamat memori dari variabel tersebut

3.

a. Code

```
#include <iostream>

using namespace std;

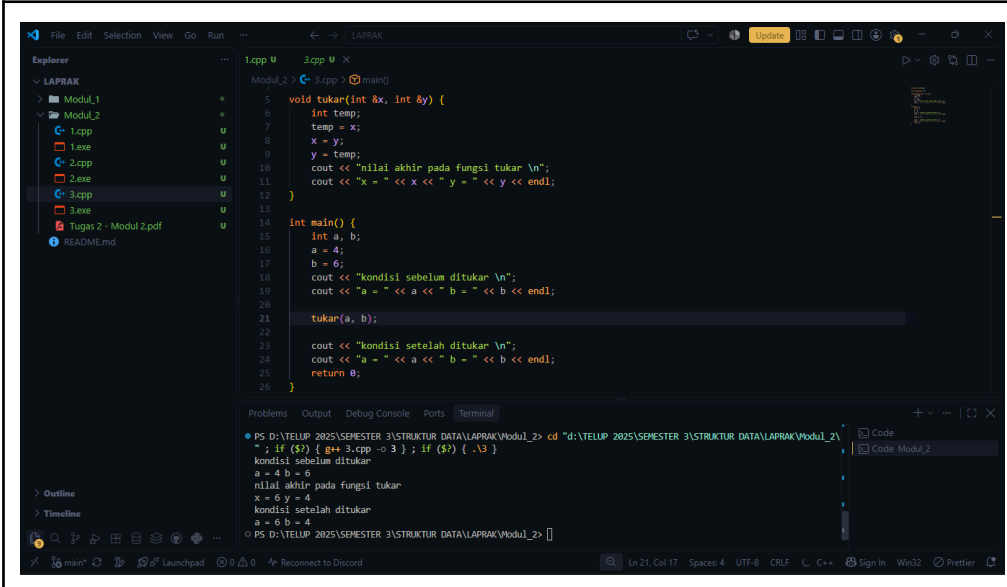
void tukar(int &x, int &y) {
    int temp;
    temp = x;
    x = y;
    y = temp;
    cout << "nilai akhir pada fungsi tukar \n";
    cout << "x = " << x << " y = " << y << endl;
}

int main() {
    int a, b;
    a = 4;
    b = 6;
    cout << "kondisi sebelum ditukar \n";
    cout << "a = " << a << " b = " << b << endl;

    tukar(a, b);

    cout << "kondisi setelah ditukar \n";
    cout << "a = " << a << " b = " << b << endl;
    return 0;
}
```

b. Screenshot output



The screenshot shows a Visual Studio Code editor with a C++ project named 'LAPRAK'. The file explorer on the left shows a folder 'Modul_2' containing files '1.cpp', '1.exe', '2.cpp', '2.exe', '3.cpp', '3.exe', 'Tugas 2 - Modul 2.pdf', and 'README.md'. The main editor displays the source code of '3.cpp', which is identical to the code provided in the previous block. The terminal at the bottom shows the output of the program after running '3.exe'. The output is as follows:

```
PS D:\TELLUP 2025\SEMESTER 3\STRUKTUR DATA\LAPRAK\Modul_2> cd "d:\TELLUP 2025\SEMESTER 3\STRUKTUR DATA\LAPRAK\Modul_2"
" ; if ($?) { g++ 3.cpp -o 3 } ; if ($?) { .\3 }
kondisi sebelum ditukar
a = 4 b = 6
nilai akhir pada fungsi tukar
x = 6 y = 4
kondisi setelah ditukar
a = 6 b = 4
```

c. Penjelasan dari code dan output (korelasi)

Program ini caranya mirip-mirip sama tujuan program kedua (sama-sama bisa ngubah nilai asli), tapi teknik yang dipakai beda, yaitu pakai reference `int &x` dan `int &y`. Kalau di pass by pointer kita masih harus pakai `&` pas manggil fungsi dan `*` buat akses nilainya, di pass by reference ini semuanya jadi lebih simpel pemanggilan fungsinya `tukar(a, b)` aja, sama kayak pass by value, ngga perlu ribet-ribet pakai simbol pointer.

Meskipun cara nulisnya lebih sederhana, secara konsep `x` dan `y` di sini itu sebenarnya cuma "nama lain" atau alias dari `a` dan `b`. Jadi begitu nilai `x` dan `y` ditukar di dalam fungsi, otomatis nilai `a` dan `b` di `main()` juga ikut ketuker, terlihat dari output akhir `a = 6` dan `b = 4`. Bisa dibilang, pass by reference ini kayak gabungan kelebihan dari dua metode sebelumnya = bisa ngubah nilai variabel asli (kayak pointer), tapi cara penulisan kodenya tetap simpel dan gampang dibaca (kayak pass by value)